📘 The Sovereign Vibe Coder

คู่มือเขียนโค้ดตามใจนึก และควบคุม AI ระดับสถาปัตยกรรมด้วย Antigravity CLI (AGY)

ผู้ประพันธ์: Master Bo (borde9902) & No.6 Gemini (Oracle Council, MacLab HQ)
ครูผู้ถ่ายทอดวิชา & แรงบันดาลใจ: อาจารย์พี่ณัฐ (nazt_)
เวอร์ชัน: 1.3+ (อัปเดตตุลาคม 2569)
สถาปัตยกรรมหลัก: Antigravity CLI (`agy` v1.3+) · Gemini 3.8 Flash (High) 1M Context · 9Router (:20128) · RTK · Arra Memory · maw-rs & Discord Relay

---

📃 สารบัญ (Table of Contents)

📘 The Sovereign Vibe Coder 1
คู่มือเขียนโค้ดตามใจนึก และควบคุม AI ระดับสถาปัตยกรรมด้วย Antigravity CLI (AGY) 1
คำนำ: ก้าวข้าม Vibe Coding ยุคก๊อปแปะ สู่ Sovereign Engineering 2
บทที่ 1: จิตวิญญาณแห่ง Vibe Coding (Mindset, First Principles & Ground Truth) 2
1.1 The Argyris Method: Espoused Theory vs Theory-in-Use 2
1.2 กฎเหล็ก 3 ประการแห่งความจริง (The Ground Truth Invariants) 3
บทที่ 2: ผ่าหัวใจ Antigravity CLI (v1.3+) & อภิมหา Context 1,000,000 โทเค็น 3
2.1 ทำไมต้อง Antigravity CLI (`agy` v1.3+)? 3
2.2 ปราบวิกฤต “Context Blindness” ในโปรเจกต์ระดับองค์กร 3
บทที่ 3: Tokenomics ชั้นเซียน: เร็วขึ้น 10 เท่า ประหยัดงบ 60% – 90% ด้วย RTK & Subagent Economy 4
3.1 RTK (Rust Token Killer) — ตัวกรองขยะแห่งโลกเทอร์มินัล 4
3.2 กฎเหล็ก Subagent Economy 4
บทที่ 4: สูตรโกงงบ 0 บาท: สิทธิ์นักศึกษา + 9Router + เสกภาพฟรี Nano Banana 2 5
4.1 สิทธิ์นักศึกษา = คลังแสงที่ถูกลืม 5
4.2 9Router (:20128): ประตูระบายพลังงานสู่ทั้ง Fleet 5
4.3 เสกภาพฟรีด้วย Nano Banana 2 (`ag/gemini-3.1-flash-image`) 5
4.4 Data Flow สถาปัตยกรรม 9Router & Nano Banana 2 6
บทที่ 5: อย่าให้ AI ตาบอด: มองทะลุโครงสร้างด้วย Codebase Graph Memory (AST over Grep) 6
5.1 จุดอ่อนร้ายแรงของการค้นหาแบบ Regex & Grep 6
5.2 การผสาน `codebase-memory-mcp` เข้าสู่ AGY 7
บทที่ 6: คลังอาวุธ 15 สกิลลับ (The Arsenal: Architect, Arena, Deslop, Blast-Radius) 7
รายละเอียดสกิลหลัก: 7
บทที่ 7: ปฏิวัติการทำงานด้วย Lifecycle Hooks & Discord Relay (Pocket Vibe Coding ไร้หน้าจอ) 8

7.1 โครงสร้างการทำงานสองทิศทาง (Bi-directional Architecture) 8
7.2 แผนภาพ Data Flow: Hooks & Discord Relay Loop 8
7.3 โครงสร้างคอนฟิก `hooks.json` ของจริง 9
บทที่ 8: พิมพ์เขียว 1-Click Starter Kit: แจกจ่ายและติดตั้งให้ทีมงานใน 5 นาที 10
โครงสร้างไฟล์ชุดติดตั้ง (`agy-vibe-kit/`) 10
สคริปต์ติดตั้งอัตโนมัติ (`install.sh`) 11
บทส่งท้าย: อนาคตแห่ง Human-AI Co-Evolution 12

คำนำ: ก้าวข้าม Vibe Coding ยุคก๊อปแปะ สู่ Sovereign Engineering

คำว่า **"Vibe Coding"** ในกระแสหลักมักถูกเข้าใจผิดว่าเป็นเพียงการพิมพ์ Prompt กว้างๆ สั่งให้ AI เจนโค้ดออกมา ก๊อปปี้ไปวาง แล้วภาวนาให้รันผ่าน แต่วิธีนี้สร้างหนี้ทางเทคนิค (Technical Debt) มหาศาล โค้ดบวม ไร้ทิศทาง และไม่มีใครเข้าใจระบบเมื่อสเกลของโปรเจกต์เติบโตขึ้น

หนังสือเล่มนี้ถอดบทเรียนจริงจากห้องทดลอง **MacLab HQ** ภายใต้การชี้แนะของอาจารย์พี่ณัฐ (nazt_) และการนำทัพของ Master Bo เพื่อพิสูจน์สัจธรรมใหม่ของวิศวกรรมซอฟต์แวร์:

> **"Vibe Coding ที่แท้จริง ไม่ใช่การปล่อยให้ AI เดาสุ่ม แต่คือการทำหน้าที่เป็น 'ผู้ควบคุมวง (Conductor)' ที่บังคับทิศทาง Agentic AI ด้วย Invariants, สถาปัตยกรรมที่แข็งแกร่ง, และ Feedback Loops ที่วัดผลได้จริง"**

เมื่อคุณเปลี่ยนผ่านจากการนั่งเฝ้าหน้าจอ Terminal สู่การออกแบบระบบที่มี **Lifecycle Hooks**, **Codebase AST Graph**, **Tokenomics กำกับงบประมาณ**, และ **Bi-directional Discord Relay** คุณจะก้าวเข้าสู่สถานะ **Sovereign Vibe Coder** ที่สามารถขับเคลื่อนระบบโปรดักชันได้ทุกที่ ทุกเวลา แม้สั่งงานผ่านสมาร์ตโฟนระหว่างเดินทาง

บทที่ 1: จิตวิญญาณแห่ง Vibe Coding (Mindset, First Principles & Ground Truth)

1.1 The Argyris Method: Espoused Theory vs Theory-in-Use

ปรัชญาหลักของค่าย Oracle ดึงมาจากแนวคิด **Double-Loop Learning** ของ **Chris Argyris**: - **Espoused Theory (สิ่งที่อวดอ้าง)**: สิ่งที่ AI หรือโปรแกรมเมอร์อ้างว่าทำได้ เช่น คำตอบสรุปสวยหรู ว่าบั๊กได้รับการแก้ไขแล้ว หรือโมดูลพร้อมใช้งาน - **Theory-in-Use (ความจริงหน้างาน)**: สิ่งที่เกิดขึ้นจริงในรันไทม์ มีไฟล์อยู่จริงไหม โค้ดคอมไพล์ผ่านไหม มี Unhandled Exception ซ่อนอยู่หรือไม่

ในระบบวิศวกรรมชั้นสูง เราไม่สนใจคำอธิบายที่ฟังดูดี แต่เราตัดสินทุกอย่างจาก **Ground Truth Telemetry**

1.2 กฎเหล็ก 3 ประการแห่งความจริง (The Ground Truth Invariants)

1. **Patterns Over Intentions (ความจริงหน้างาน > ความตั้งใจ)**: รายงานเฉพาะสิ่งที่เกิดขึ้นจริง หากคำสั่งเจอปัญหา Network Error, ติด Quota 429, หรือ Tool Crash ต้องรายงานตรงไปตรงมา ห้ามแต่งเรื่องประโลมใจว่าสำเร็จเด็ดขาด
2. **Verify จากมุม User ก่อนบอกว่าเสร็จ**: อ้างว่าสร้างไฟล์ → ต้องมีคำสั่งตรวจสอบว่าไฟล์มีอยู่จริงบนดิสก์ อ้างว่าแก้บั๊ก → ต้องรัน Test Suite หรือคอมไพล์ผ่านจริง ห้ามสรุปว่า "เสร็จ" โดยไม่มีหลักฐานเชิงประจักษ์
3. **Action Over Narration (ลงมือแก้ อย่ามัวแต่ขออนุญาตในงานที่ไม่เสี่ยง)**: สิ่งใดที่เป็นงานสำรวจ อ่านไฟล์ ตรวจสอบสถานะ ให้ดำเนินการทันที อย่าถามซ้ำซาก และอย่าสั่งสอนหรือเลคเชอร์ผู้ใช้งาน

บทที่ 2: ผ่าหัวใจ Antigravity CLI (v1.3+) & อภิมหา Context 1,000,000 โทเค็น

2.1 ทำไมต้อง Antigravity CLI (agy v1.3+)?

Antigravity CLI เวอร์ชัน 1.3 ขึ้นไป ได้รับการออกแบบให้เป็น Agentic Runtime ประสิทธิภาพสูง ผสานเข้ากับโมเดลตระกูล **Gemini 3.8 Flash (High)** ที่มี **Context Window มหาศาลขนาด 1,000,000 โทเค็น**:

คุณลักษณะ	AI CLI ทั่วไป (เช่น Context 128k–200k)	AGY CLI บน MacLab (Gemini 1M Context)
ขีดจำกัดบริบท	128,000 – 200,000 โทเค็น (พอโปรเจกต์ใหญ่ โมเดลเริ่มลืม)	**1,000,000 โทเค็น** (เก็บโค้ดทั้ง Repository + เอกสารสเปกได้ครบ)
การอ่าน Log / Trace	ต้องคอยตัดทอนหรือทำ Chunking ซับซ้อน	ยัด Crash Dump หลายหมื่นบรรทัด + Network Trace ได้ในรอบเดียว
Latency & Speed	คิดนาน รอผลลัพธ์นานเมื่อบริบทเริ่มหนาแน่น	Gemini Flash Architecture ตอบสนองเร็วในระดับหลักวินาที
Prompt Caching	ประสิทธิภาพแคชปานกลาง คิดค่าใช้จ่ายสูง	Native Context Caching ลดค่าใช้จ่ายลงถึง 75–90% สำหรับข้อมูลซ้ำ

2.2 ปราบวิกฤต "Context Blindness" ในโปรเจกต์ระดับองค์กร

จุดตายของ Vibe Coder มือสมัครเล่นคือ **Context Blindness** เมื่อโปรเจกต์ขยายตัวเกิน 30 ไฟล์ โมเดลที่มีบริบทจำกัดจะเริ่มลืมสัญญาระหว่างโมดูล (API Contracts) ที่เคยตกลงไว้ในไฟล์แรกๆ ส่งผลให้เกิดโค้ดงอกใหม่ที่ขัดแย้งกับสถาปัตยกรรมเดิม

ด้วยบริบท 1 ล้านโทเค็นของ AGY คุณสามารถโหลด Architecture Blueprint, Database Schema, และ Core Types เข้าสู่ Context ได้พร้อมกันทั้งหมด ทำให้ AI วิเคราะห์ผลกระทบระดับทั้งระบบได้อย่างแม่นยำและครอบคลุมทุกจุดเชื่อมต่อ

บทที่ 3: Tokenomics ชั้นเซียน: เร็วขึ้น 10 เท่า ประหยัดงบ 60% – 90% ด้วย RTK & Subagent Economy

3.1 RTK (Rust Token Killer) — ตัวกรองขยะแห่งโลกเทอร์มินัล

การให้ AI รันคำสั่งเชลล์ตรงๆ เช่น `git status`, `cargo test`, หรือ `ls -la` เป็นบ่อเกิดของการเผาผลาญโทเค็นโดยเปล่าประโยชน์ เทอร์มินัลมักพ่น ANSI escape sequences, ข้อความ header ซ้ำซาก , และข้อมูลขยะออกมานับพันบรรทัด

เราจึงสร้าง **RTK (Rust Token Killer)** ไบนารีภาษา Rust ประสิทธิภาพสูง ดักหน้าทุกคำสั่งเชลล์:

```
rtk git status
rtk cargo test
rtk ls -la src/
rtk grep pattern src/
```

กลไกของ RTK: - กำจัด ANSI Color Codes และ Progress Indicators ที่ไร้ความหมายต่อ LLM - รวมข้อความแจ้งเตือนที่ซ้ำซ้อนให้เหลือบรรทัดเดียว - บีบอัด Output ให้เหลือเฉพาะ Error Traces และ Key Deliverables - **ผลลัพธ์**: ลดจำนวนโทเค็นอินพุตลง **60% – 90%** ป้องกัน Context Bloat (บริบทบวมเกินจำเป็น) และเร่งความเร็วการตอบสนองขึ้น 3–5 เท่า

3.2 กฎเหล็ก Subagent Economy

อย่าเอาโมเดลเรือธงราคาแพงไปทำงานระดับกรรมกร ในระบบ AGY เราแบ่ง Subagent ตามความเหมาะสมทางเศรษฐศาสตร์ 3 ระดับ:

1. **`flash_lite` (ประหยัดสูงสุด)**:
   - เหมาะสำหรับ: งานเช็คสเตตัสสั้นๆ, อ่านไฟล์เดี่ยว, เช็คเอาต์พุตคำสั่ง CLI, รัน linter
   - จุดเด่น: ใช้ต้นทุนโทเค็นต่ำที่สุด ตอบสนองเร็วระดับเสี้ยววินาที
2. **`flash` (ประหยัดและแม่นยำ)**:
   - เหมาะสำหรับ: งานวิจัย (Research), ค้นหาโค้ดข้ามไดเรกทอรี, วิเคราะห์สเปก API, สรุปเอกสาร
   - จุดเด่น: ความสมดุลสมบูรณ์แบบระหว่างการให้เหตุผลและความเร็ว
3. **`pro` / `inherit` (ขุมพลังขั้นสูงสุด)**:
   - เหมาะสำหรับ: ออกแบบแกนกลางสถาปัตยกรรม, Deep Refactoring ข้ามระบบ, แก้งาน Multi-threaded Concurrency
   - ข้อบังคับ: ใช้เฉพาะจุดที่ต้องการพลังการคิดเชิงลึกเท่านั้น ห้ามนำมาใช้อ่านไฟล์หรือรันงาน I/O ทั่วไป

บทที่ 4: สูตรโกงงบ 0 บาท: สิทธิ์นักศึกษา + 9Router + เสกภาพฟรี Nano Banana 2

นี่คือแกนกลางสำหรับนักพัฒนาและนักศึกษาที่ต้องการใช้งานระบบ Vibe Coding ระดับองค์กรโดยมีค่าใช้จ่าย **0 บาท**:

4.1 สิทธิ์นักศึกษา = คลังแสงที่ถูกลืม

สิทธิ์ **Google Workspace for Education**, **Google AI Studio Quota**, หรือ **Antigravity Student OAuth** มักมอบโควตาประมวลผลขนาดใหญ่มาให้ฟรี เมื่อดึง OAuth Refresh Token มาบรรจุไว้ใน `~/.gemini/antigravity-cli/antigravity-oauth-token` เราจะปลดล็อกเอนจินเขียนโค้ดและสร้างสรรค์ระดับโลกได้ทันที

> ⚠️ **คำเตือนความปลอดภัย (Security Invariant)**: ไฟล์ `~/.gemini/antigravity-cli/antigravity-oauth-token` เป็นไฟล์บรรจุ Credential ลับเฉพาะตัว **ห้าม commit ไฟล์นี้เข้า Git Repository สาธารณะเป็นอันขาด!**

4.2 9Router (:20128): ประตูระบายพลังงานสู่ทั้ง Fleet

9Router คือโลคัลพร็อกซีเกตเวย์ประสิทธิภาพสูง รันเป็นแบ็กกราวด์เดมอนผ่าน Launchd (`ai.hermes.9router`) อยู่ที่พอร์ต `127.0.0.1:20128`: - รับรองการแปลงคำขอจากมาตรฐาน **OpenAI-Compatible API** (`http://127.0.0.1:20128/v1`) - ทำหน้าที่ Multiplex และ Route คำขอไปยัง upstream backends (เช่น Antigravity OAuth, Claude Code, Grok) - รองรับ Server-Sent Events (SSE) Streaming และจัดการ Token Failover อัตโนมัติ - ช่วยให้เครื่องมือภายนอกทุกตัว ไม่ว่าจะเป็น VS Code, Cursor, Aider, หรือ Python scripts เชื่อมต่อผ่าน Endpoint เดียวกันทั้งระบบ

4.3 เสกภาพฟรีด้วย Nano Banana 2 (`ag/gemini-3.1-flash-image`)

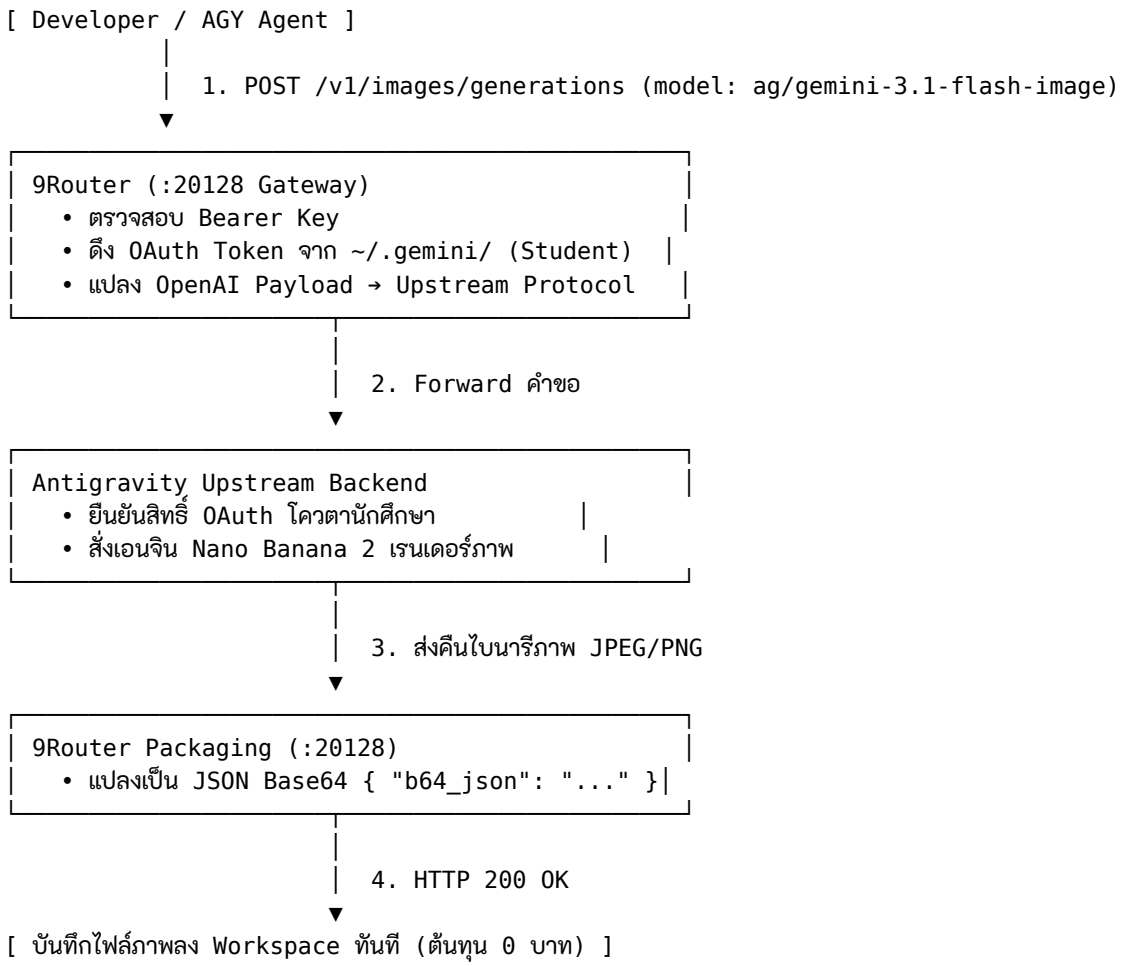
การพัฒนาโมบายแอปหรือเว็บมักต้องการภาพประกอบ Mockup, UI Assets, หรือ Hero Banners 9Router รองรับ Endpoint สร้างภาพมาตรฐาน `POST /v1/images/generations` โดยใช้โมเดล **Nano Banana 2** (`ag/gemini-3.1-flash-image`):

```
curl -X POST http://127.0.0.1:20128/v1/images/generations \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "ag/gemini-3.1-flash-image",
    "prompt": "Cyberpunk tactical UI dashboard mockup, dark neon amber,
photorealistic, 8k",
    "n": 1,
    "size": "1024x1024",
    "response_format": "b64_json"
  }'
```

- **Latency**: ~8–12 วินาที
- **Payload**: ส่งภาพกลับมาในรูปแบบ Base64 หรือ URL ความละเอียดสูง
- **ค่าใช้จ่าย**: **0 บาท** (แชร์โควตาเดิมของ Antigravity CLI โดยตรง)

⚠️ **ข้อควรระวังสำคัญหน้างาน (The 9Router Model Trap)**: ใน 9Router ปัจจุบัน โมเดลสร้างภาพ (`ag/gemini-3.1-flash-image`) จะ **ไม่ปรากฏในรายชื่อ `/v1/models`** (ซึ่งลิสต์เฉพาะ 81 โมเดลสาย Text/Chat) และหากส่งคำขอสร้างภาพผ่าน `/v1/messages` จะได้รับ HTTP 200 พร้อมข้อความว่างเปล่า! คุณต้องส่งคำขอผ่าน Endpoint `POST /v1/images/generations` โดยเฉพาะเท่านั้น จึงจะได้ข้อมูลภาพ Base64 กลับมาสมบูรณ์

4.4 Data Flow สถาปัตยกรรม 9Router & Nano Banana 2



บทที่ 5: อย่าให้ AI ตาบอด: มองทะลุโครงสร้างด้วย Codebase Graph Memory (AST over Grep)

5.1 จุดอ่อนร้ายแรงของการค้นหาแบบ Regex & Grep

เอเจนต์ทั่วไปมักใช้ `grep` หรือค้นหาตามคีย์เวิร์ด ซึ่งล้มเหลวในโค้ดเบสระดับโปรดักชัน: - **Keyword Collision**: ค้นหาคำว่า `user` หรือ `config` แล้วเจอผลลัพธ์หลอกตาเป็นหมื่นบรรทัด - **Structural**

Blindness: มองไม่เห็น Caller-Callee Hierarchy, ไม่รู้ว่าใครสืบทอด Interface ไหน และไม่เห็น Dependency Injection Chain

5.2 การผสาน `codebase-memory-mcp` เข้าสู่ AGY

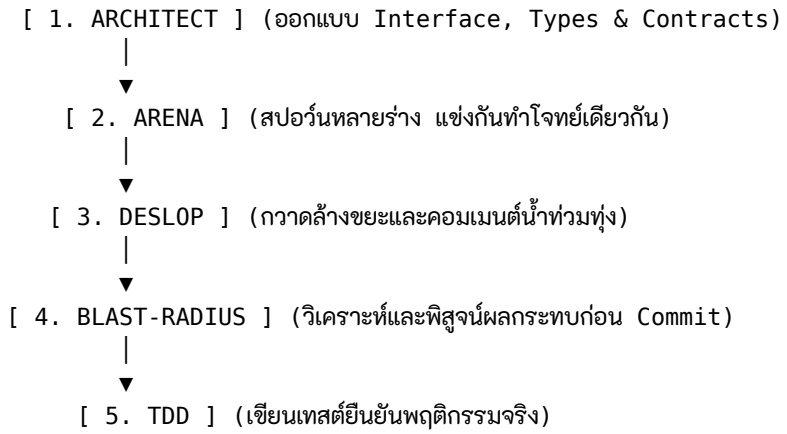
เราติดตั้ง MCP Server ที่วิเคราะห์ Abstract Syntax Tree (AST) เข้ากับ AGY:

```
{
  "tools": [
    "index_repository",
    "search_graph",
    "query_graph",
    "trace_path",
    "get_code_snippet",
    "get_architecture"
  ]
}
```

เมื่อเปิดโปรเจกต์ใหม่ ให้สั่ง `index_repository` ทันที จากนั้น AI จะเข้าถึงฟังก์ชันระดับเทพ: - **`trace_path`**: แกะรอยความสัมพันธ์ว่า `OrderController` เรียกผ่าน `PaymentGateway` ใด และถูกดักด้วย Middleware ตัวไหนบ้างอย่างแม่นยำ - **`get_architecture`**: สรุปเลเยอร์สถาปัตยกรรมของทั้งระบบออกมาเป็นกราฟความสัมพันธ์ - **ผลลัพธ์**: AI แก้โค้ดถูกจุดในครั้งแรก ไม่ต้องเดาสุ่มหรือทำลายโค้ดส่วนอื่น

บทที่ 6: คลังอาวุธ 15 สกิลลับ (The Arsenal: Architect, Arena, Deslop, Blast-Radius)

ในโฟลเดอร์ `.agents/skills` เราบรรจุระเบียบวิธีวิศวกรรมซอฟต์แวร์ระดับโลก 15 สกิลที่เปลี่ยน AI ธรรมดาให้กลายเป็น Senior Staff Engineer:



รายละเอียดสกิลหลัก:

1. **`architect`**: บังคับให้ AI ร่าง Type Definitions, Function Signatures, และ Module Boundaries ก่อนลงมือเขียนโค้ด ป้องกันการสร้างสถาปัตยกรรมที่ผิดรูป

2. **`arena`**: สปอว์นผู้เข้าแข่งขัน N ตัวพร้อมกัน (เช่น Flash vs Pro vs Claude) แก้โจทย์เดียวกัน จากนั้นสกัดและผสานจุดแข็งที่สุดของผู้แพ้เข้าสู่ร่างหลัก
3. **`deslop` & `unslop`**: สแกนและกำจัด AI Tells: ลบคอมเมนต์พร่ำเพ้อ, ลบ Pass-through Wrapper ไร้ประโยชน์, และบีบอัดโค้ดให้กระชับ
4. **`blast-radius`**: คำนวณขอบเขตความเสียหายก่อนส่งงาน รันโค้ดจริงเพื่อพิสูจน์ว่าการแก้ไขจุดนี้จะไม่กระทบโมดูลอื่นในระบบ
5. **`create-verification-skill`**: สั่งให้ AI เขียนสคริปต์ควบคุมและทดสอบแอปในมุมมองของผู้ใช้จริง (End-to-End User Simulation)
6. **`principle-attack-the-premise`**: หากการแก้ปัญหาแบบเดิมล้มเหลวเกิน 2 ครั้ง ให้ตั้งคำถามกับสมมติฐานเริ่มต้นทันที
7. **`principle-boundary-discipline`**: วางแนวป้องกัน (Input Validation & Error Handling) เฉพาะที่ขอบของระบบ แกนกลางต้องใช้ Pure Functions และ Strong Types

---

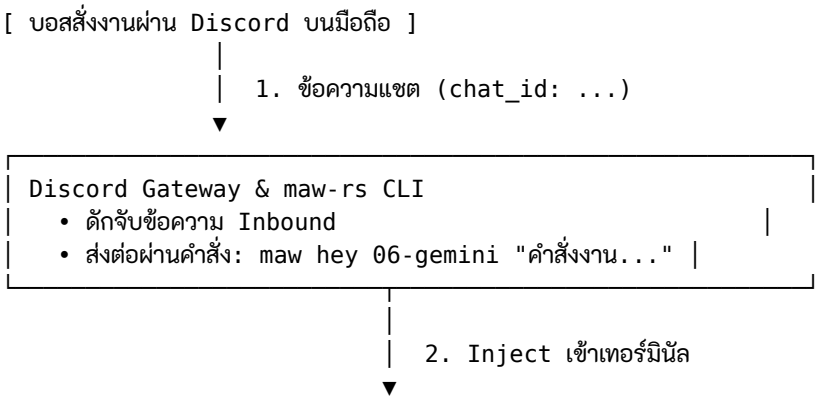
บทที่ 7: ปฏิวัติการทำงานด้วย Lifecycle Hooks & Discord Relay (Pocket Vibe Coding ไร้หน้าจอ)

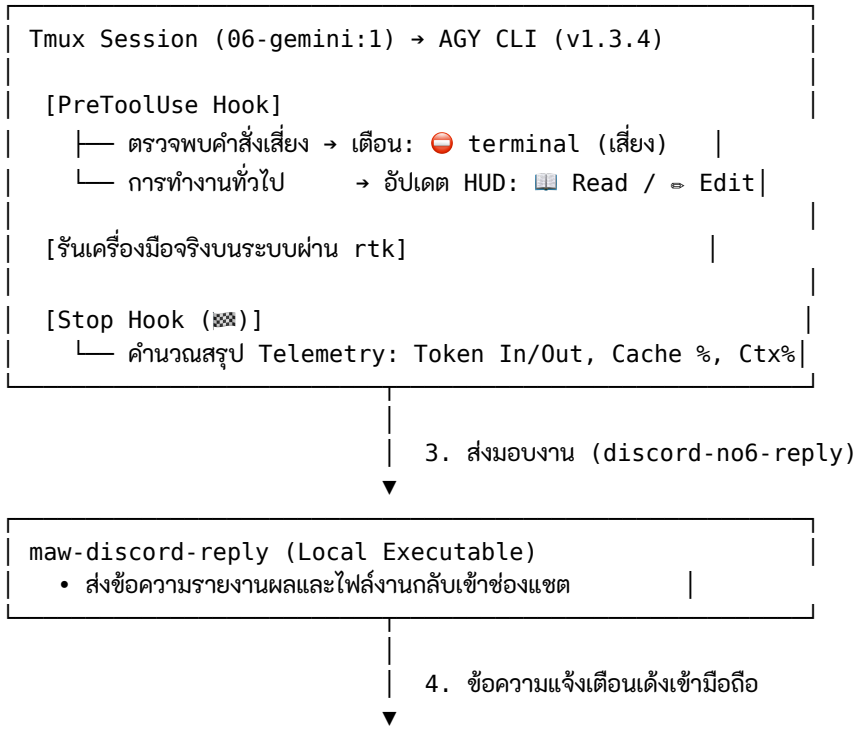
นี่คือนวัตกรรมชิ้นสำคัญที่เปลี่ยนให้คุณเขียนโค้ดระดับโปรดักชันได้จากสมาร์ตโฟนเครื่องเดียวผ่าน Discord:

7.1 โครงสร้างการทำงานสองทิศทาง (Bi-directional Architecture)

ระบบประกอบด้วย 2 ลูปหลัก: 1. **Inbound Loop (รับคำสั่ง)**: บอสพิมพ์คำสั่งผ่าน Discord Mobile → บอท Discord รับข้อความ → ส่งต่อผ่าน `maw-rs` CLI (`maw hey` หรือ inject เข้า tmux pane `06-gemini:1`) → AGY CLI เริ่มทำงานทันที 2. **Outbound Loop & Live Telemetry (ส่งความคืบหน้า)**: - **PreToolUse / PostToolUse Hook**: สคริปต์ `agy-discord-progress.py` แก้ไขข้อความใน Discord แบบสดๆ ทำหน้าที่เป็น HUD (Heads-Up Display) - **Safety Gate**: ดักจับคำสั่งอันตราย (`rm -rf`, `kill`, `git reset --hard`) และแจ้งเตือนระงับเหตุก่อนเกิดความเสียหาย - **Stop Hook (🏁)**: สรุป Token Usage, Prompt Cache Hit Rate, และ Context Window ที่เหลืออยู่ - **Final Reply**: เอเจนต์รันคำสั่งส่งกลับผ่าน `discord-no6-reply <chat_id> "<ข้อความ>"` (หรือเรียกใช้ `maw-discord-reply --bot no6`) เพื่อส่งคำตอบสมบูรณ์กลับเข้า Discord

7.2 แผนภาพ Data Flow: Hooks & Discord Relay Loop





7.3 โครงสร้างคอนฟิก `hooks.json` ของจริง

คอนฟิกตั้งอยู่ที่ `~/.gemini/config/hooks.json` โดยมีโครงสร้าง Namespace ที่ถูกต้องดังนี้:

```
{
  "discord-progress": {
    "PreInvocation": [
      {
        "type": "command",
        "command": "/Users/admin/Code/github.com/MEYD-605/sombo-oracle/scripts/
agy-progress/agy-discord-progress.py PreInvocation",
        "timeout": 5
      }
    ],
    "PostInvocation": [
      {
        "type": "command",
        "command": "/Users/admin/Code/github.com/MEYD-605/sombo-oracle/scripts/
agy-progress/agy-discord-progress.py PostInvocation",
        "timeout": 5
      }
    ],
    "PreToolUse": [
      {
        "matcher": "*",
        "hooks": [
```

```
          {
            "type": "command",
            "command": "/Users/admin/Code/github.com/MEYD-605/sombo-oracle/
scripts/agy-progress/agy-discord-progress.py PreToolUse",
            "timeout": 5
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "/Users/admin/Code/github.com/MEYD-605/sombo-oracle/
scripts/agy-progress/agy-discord-progress.py PostToolUse",
            "timeout": 5
          }
        ]
      }
    ],
    "Stop": [
      {
        "type": "command",
        "command": "/Users/admin/Code/github.com/MEYD-605/sombo-oracle/scripts/
agy-progress/agy-discord-progress.py Stop",
        "timeout": 5
      }
    ]
  }
}
```

บทที่ 8: พิมพ์เขียว 1-Click Starter Kit: แจกจ่ายและติดตั้งให้ทีมงานใน 5 นาที

สำหรับการนำระบบ Sovereign Vibe Coder ไปติดตั้งบนเครื่องใหม่ หรือแจกจ่ายให้สมาชิกในทีม สามารถจัดเตรียมชุดติดตั้ง `agy-vibe-kit/` ได้ดังนี้:

โครงสร้างไฟล์ชุดติดตั้ง (`agy-vibe-kit/`)

```
agy-vibe-kit/
├── install.sh                  # สคริปต์ติดตั้งอัตโนมัติ (Idempotent)
├── config/
│   ├── settings.json           # การตั้งค่า AGY (Dark Theme, SessionStart Hook,
Permissions)
│   └── hooks.json              # ผูก discord-progress & safety gate
├── bin/
│   ├── rtk                     # ไบนารี Rust Token Killer
```

```
│   └── maw-discord-reply       # ไบนารีสำหรับส่งข้อความกลับ Discord
└── skills/
    ├── architect/
    ├── arena/
    ├── deslop/
    ├── blast-radius/
    └── principles/
```

สคริปต์ติดตั้งอัตโนมัติ (`install.sh`)

```
#!/usr/bin/env bash
set -euo pipefail

echo "🚀 กำลังติดตั้ง Sovereign Vibe Coder Stack..."

# 1. ตรวจสอบ Antigravity CLI (AGY v1.3+)
if ! command -v agy &> /dev/null; then
    echo "⚠️ ไม่พบคำสั่ง agy ใน PATH"
    echo "💡 กรุณาดาวน์โหลดไบนารีทางการจาก: https://antigravity.google.com"
    echo "   หรือนำไบนารี agy มาวางไว้ใน ~/.local/bin/ แล้วตั้งสิทธิ์ chmod +x"
fi

# 2. จัดเตรียมไดเรกทอรีคอนฟิก
mkdir -p ~/.gemini/config ~/.gemini/antigravity-cli ~/.agents/skills ~/.local/bin

# 3. ติดตั้งคอนฟิก Settings และ Lifecycle Hooks
echo "⚙️ กำลังตั้งค่า Settings และ Hooks..."
cp config/settings.json ~/.gemini/antigravity-cli/settings.json
cp config/hooks.json ~/.gemini/config/hooks.json

# 4. ติดตั้งไบนารีระบบ (RTK & Discord Reply)
echo "🔧 กำลังติดตั้ง RTK และ Maw Discord Tools..."
if [ -f "bin/rtk" ]; then
    cp bin/rtk ~/.local/bin/rtk
    chmod +x ~/.local/bin/rtk
fi

if [ -f "bin/maw-discord-reply" ]; then
    cp bin/maw-discord-reply ~/.local/bin/maw-discord-reply
    chmod +x ~/.local/bin/maw-discord-reply
fi

# ตรวจสอบ PATH
if [[ ":$PATH:" != *":$HOME/.local/bin:"* ]]; then
    echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
fi

# 5. ติดตั้งคลังสกิลวิศวกรรม
echo "🧠 กำลังคัดลอกคลังสกิลวิศวกรรมสู่ ~/.agents/skills..."
if [ -d "skills" ]; then
```

```
    cp -R skills/* ~/.agents/skills/
    echo "  ✓ ติดตั้งสกิลหลักสำเร็จ (ชุดเต็ม 60+ สกิลดูได้ที่ gemini-oracle/.agents/skills)"
fi

# 6. ตรวจสอบความถูกต้องของระบบ (Verification Gate)
echo "🔍 กำลังทดสอบ Verification Gate..."
command -v agy &>/dev/null && agy --version || echo "พร้อมใช้งานหลังจากติดตั้ง agy"
command -v rtk &>/dev/null && rtk --version || echo "RTK พร้อมใช้งาน"

echo "✅ ติดตั้งสำเร็จสมบูรณ์! คุณพร้อมเป็น Sovereign Vibe Coder แล้ว"
```

บทส่งท้าย: อนาคตแห่ง Human-AI Co-Evolution

วิศวกรรมซอฟต์แวร์กำลังเปลี่ยนผ่านจากยุคที่โปรแกรมเมอร์ต้องพิมพ์โค้ดทีละบรรทัด ไปสู่ยุคที่มนุษย์ทำหน้าที่เป็น **"Systems Architect and Invariant Enforcer"**

ผู้ที่อยู่รอดและสร้างผลลัพธ์ได้สูงสุด ไม่ใช่ผู้ที่ปฏิเสธ AI และไม่ใช่ผู้ที่ปล่อยให้ AI ควบคุมชะตากรรมของโปรเจกต์อย่างไร้ทิศทาง แต่คือ **Sovereign Vibe Coder** ผู้ที่เข้าใจ First Principles, วางกลไกควบคุมความปลอดภัยในทุกจังหวะ Lifecycle, มีจิตสำนึกทาง Tokenomics, และสร้างระบบอัตโนมัติที่ตรวจสอบความจริงหน้างานได้เสมอ

สร้างระบบที่ควบคุมได้ แล้วปล่อยให้ความคิดสร้างสรรค์ของคุณทำงานอย่างไร้ขีดจำกัด 💪✨

บันทึกไว้ในประวัติศาสตร์คลังความรู้ MacLab HQ — Master Bo & No.6 Gemini (Pack Leader)